



What is a package?

A **package** is a container for the classes that are used to keep the class name space compartmentalized

Example:

You can contain all classes related to all sorting programs in your own package.



Why packages?

It allows flexibility to give same name but to many classes, that is to avoid **name space collision**.

The packages in **Java** provides mechanism for partitioning the class name space into more manageable chunks.

In fact, package is both a **naming** and a **visibility** control mechanism.

It supports **reusability** and **maintainability**.



Advantages of Packages

Packages provide **code reusability**, because a package contains group of classes.

It helps in **resolving naming collision** when multiple packages have classes with the same name.

Package also provides the **hiding of class** facility. Thus, other programs cannot use the classes from hidden package.

Access limitation can be applied with the help of packages.

Nesting of a package, that is, one package can be defined in another package in a **hierarchy fashion**.



Built-in packages in Java

Packages are nothing more than the way we organize files into different directories according to their functionality, usability as well as category they should belong to . A **Java package** is a **Java** programming language mechanism for organizing classes into name spaces.

In **Java**, already many predefined packages are available, those are to help programmers to develop their software in an easy way.

Example:

javax.swing is a package in **Java** providing all basic supports in developing GUI programs.

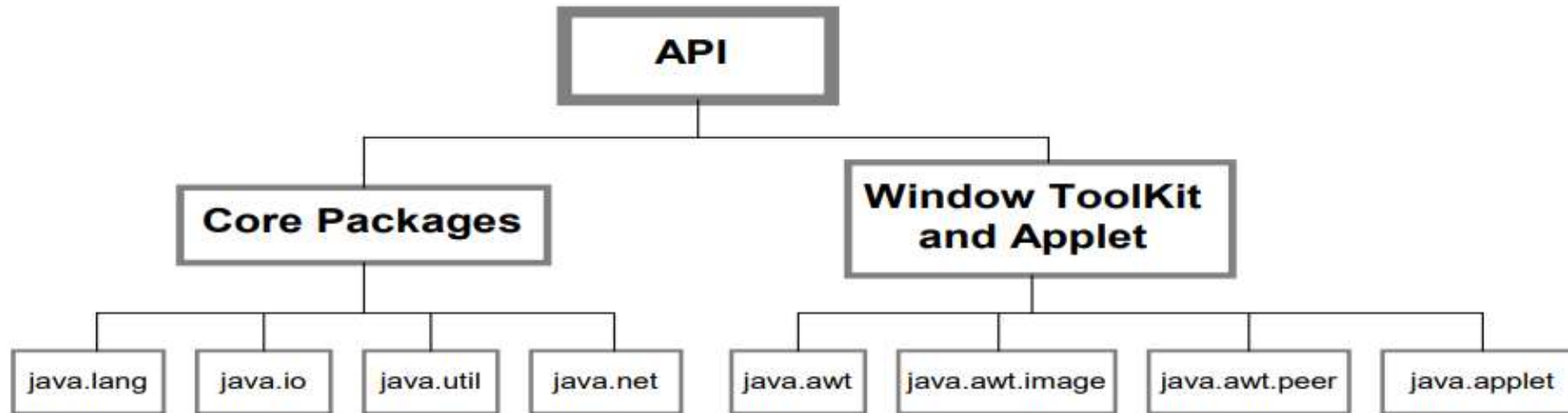


Packages in Java

- Code reusability is the main philosophy of Object-Oriented Programming.
- To power this advantage, Java has a number of packages called API bundled with the JDK.
- Packages are collection of classes and interfaces to facilitate a number of ready made solutions.
- A great knowledge of packages helps a Java developer to master in Java solution.
- In addition to the API, user can maintain their own packages.



API – The Built-in Java packages



`javax.swing` is the latest addition in Java version 5 and above.



Using API packages

- A package is a collection of classes and each class is a collection of members and methods.
- Any class as well as any member and method in a package are accessible from a Java program
- This can be achieved in Java by `import` statement.

There are
two ways of
using
`import`
statement

With fully quantified
class name

- When it is required to access a particular class
- Example: `java.lang.String`

With default (.*)
quantification

- When it is required to access a number of classes
- Example: `java.lang.*`



Using API packages

However, instead of importing whole package or a class it is possible to refer a class in order to instantiate an object.

Example:

```
java.util.Date today = new java.util.Date ( );  
System.out.println(today);
```

The same thing but with **import statement** can be done as follows

```
import java.util.Date;  
Date today = new Date( );
```



User defined packages

- User can maintain their own package.
- Java uses file system directories to store packages.
- The **package** statement can be used for the purpose with the following syntax

```
package myPackage;  
public Class myClass {  
    . . .  
    . . .  
}
```

Here, `myPackage` is the name of the package and it contains the class `myClass`.



Package naming conventions

- Packages are usually defined using a **hierarchical naming pattern**, with levels in the hierarchy separated by **periods (.)**.
- Although packages lower in the naming hierarchy are often referred to a "**sub packages**" of the corresponding packages higher in the hierarchy, there is no semantic relationship between packages.



Organizational package naming conventions

- Package names should be **all lowercase characters** whenever possible.
- Frequently, a package name begins with the top level domain name of the organization and then the organization's domain and then any subdomains listed in the reverse order.
- The organization can then choose a specific name for their package.



User defined packages

In addition to this, following steps must be taken into consideration

- Use package statement at the beginning of the package file.
- Define the class that is to be put in the package and declare it as public.
- Create a subdirectory under the working directory with the same name as the package name.
- Store the file with the same name as the `className.java` in the subdirectory created.
- Store the `compiled version` (i.e., `.class`) file into the same sub-directory.



Package design guidelines

Design guideline package cohesion

- Only **closely related classes** should belong to the same package.
- Classes that change together should belong to the same package.
- Classes that are not reused together should not belong to the same package.



Package declaration

Package declaration is file based

- All classes in the same source file belong to the same package.
- Each source file may contain an optional package declaration in the following form.

```
package <PackageName>;
```

- Let us consider the source file `ElevatorFrame.java`, for example.

```
package elevator;  
public class ElevatorFrame {  
    public double x;  
    //.....  
}
```



Package declaration

- The package declaration at the top of the source file declares that the `ElevatorFrame` class belongs to the package named `elevator`.
- When the package declaration is absent from a file, all the classes contained in the file belong to unnamed package.
- A class in a named package can be referred in two ways.



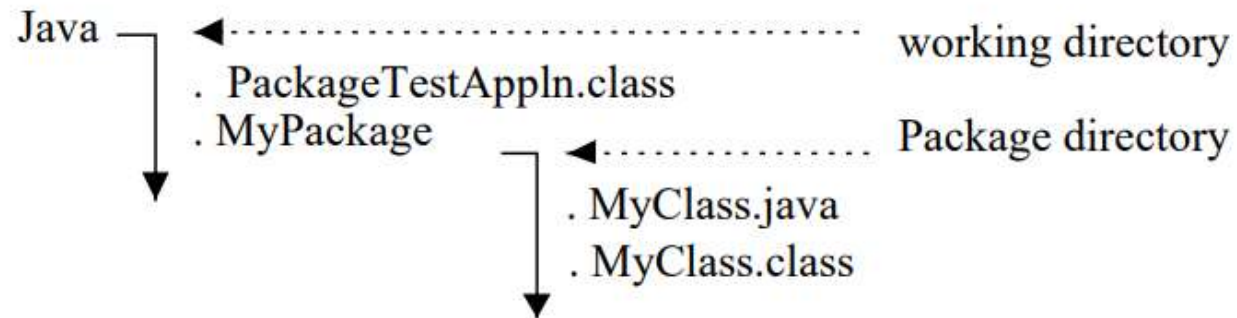
User defined packages: Example

```
// User defined package in a subdirectory, say MyPackage
package MyPackage;
public class MyClass {
    public void test ( ) {
        System.out.println ( " Welcome to My Class !");
    }
}

/* Import the package with the following code. From another distant file/
program */
import MyPackage.MyClass;
class PackageTestAppln {
    public static void main ( String args [ ] ) {
        MyClass theClass = new MyClass ( );
        theClass.test ( );
    }
}
```



User defined packages: Example





More on user defined packages

Note:

We cannot put two or more public classes together in a .java file; otherwise there will be an ambiguity in naming the .java file

? How a package then contains multiple classes?

Solution:

```
package P;  
public class A {  
    . . .  
}
```

```
package P;  
public class B {  
    . . .  
}
```



More on user defined packages

```
package P;  
public class A {  
    . . .  
}
```

```
package P;  
public class B {  
    . . .  
}
```

Steps

1. Create a directory named **P**.
2. Store the `class A` in the file `A.java` in it
3. Compile `A.java` and place it in the directory **P**.
4. Store the `class B` in the file `B.java` in it
5. Compile `B.java` and place it in the directory **P**.
6. `import P.*;`
will import all classes in the package **P**.



Finding packages and CLASSPATH

How does the Java run-time system know where to look for packages that you create?

The answer has three parts

- First, by default, the Java run-time system uses the current working directory as its starting point. Thus, if your package is in a sub-directory of the current directory, it will be found.
- Second, you can specify a directory path or paths by setting the CLASSPATH environmental variable.
- Third, you can use the -classpath option with `java` and `javac` to specify the path to your classes.



Finding packages and CLASSPATH: Example

Consider the following package specification:

```
package MyPack;
```

For a program
to find
MyPack, one
of three things
must be true.

The program can be executed from a directory
immediately above *MyPack*

The CLASSPATH must be set to include the path to
MyPack

The -classpath option must specify the path to
MyPack when the program is run via `java`.

When the second two options are used, the class
path must not include *MyPack*, itself. It must
simply specify the path to *MyPack*.

Example: In a Windows environment, if the path to
MyPack is

`C:\MyPrograms\Java\MyPack`

then the class path to *MyPack* is

`C:\MyPrograms\Java`



Finding packages and CLASSPATH: Example

```
package MyPack;
class Balance {
    String name;
    double bal;
    Balance(String n, double b) {
        name = n;
        bal = b;
    }
    void show() {
        if(bal<0)
            System.out.print("Account is dead");
        else
            System.out.print(name+": $ "+bal);
    }
}
```

```
class AccountBalance {
    public static void main(String args[]) {
        Balance current[] = new Balance[3];
        current[0] = new Balance("K. J. Fielding", 123.23);
        current[1] = new Balance("Will Tell", 157.02);
        current[2] = new Balance("Tom Jackson", -12.33);
        for(int i=0; i<3; i++)
            current[i].show();
    }
}
```



Finding packages and CLASSPATH: Example

Call this file *AccountBalance.java* and put it in a directory called *MyPack*.

Then, try executing the *AccountBalance* class, using the following command line:
`java MyPack.AccountBalance`

Next, compile the file. Make sure that the resulting *.class* file is also in the *MyPack* directory.

Remember, you will need to be in the directory above *MyPack* when you execute this command.

As explained, *AccountBalance* is now part of the package *MyPack*. This means that it cannot be executed by itself. That is, you cannot use this command line:

```
java AccountBalance
```



Importing a package

This is the general form of the import statement:

```
import pkg1 [.pkg2].(<classname> | *);
```

Here, **pkg1** is the name of a top-level package

pkg2 is the name of a sub-ordinate package inside the outer package separated by a dot (.).

There is no practical limit on the depth of a package hierarchy, except that imposed by the file system.

Finally, you specify either an explicit class name or a star (*), which indicates that the Java compiler should import the entire package.

Example:

```
import java.util.Date;  
import java.io.*;
```



Using package

Class in a named package can be referred to in two different ways

Import a class or all the classes in the designated package using

```
import PackageName.<ClassName>;  
import Packagename.*;
```

Example

The `ElevatorPanel` class in package `elevator` can simply be referred to as `elevator` when either of the following import clauses occurs at the top of source file

```
import elevator.ElevatorPanel;  
import elevator.*;
```



Access protection

Because of the interplay between classes and packages, Java addresses four categories of visibility for class members:

- Sub classes in the same package.
- Non-sub classes in the same package.
- Sub classes in different packages.
- Classes that are neither in the same package nor sub classes.

The three access modifiers, `private`, `public`, and `protected`, provide a variety of ways to produce the many levels of access required by these categories.



Access protection

	Private	No Modifier	Protected	Public
Same class	Yes	Yes	Yes	Yes
Same package subclass	No	Yes	Yes	Yes
Same package non-subclass	No	Yes	No	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes



Access protection: An example

```
//This program shows all combinations of the access control modifiers  
//In this program, we define two packages and five classes.
```

```
MyPackage1:
```

```
    class X
```

```
    class Y extends X
```

```
    class A
```

```
MyPackage2:
```

```
    class Z extends X
```

```
    class B
```



Access protection: An example

```
// Defining package MyPackagel
package MyPackagel;

public class X {
    int n = 1;
    private int p = 2;
    protected int q = 3;
    public int r = 4;
    // A constructor of the class protection
    public X() {
        system.out.println("I am constructor from class X:");
        system.out.println("n="+n);
        system.out.println("p="+p);
        system.out.println("q="+q);
        system.out.println("r="+r);
    }
}

//Save this as X.java in Mypackagel directory
```



Access protection: An example

```
package MyPackage1
class Y extends X {
    Y() {
        system.out.println("I am constructor from class Y:");
        system.out.println("n="+n);
        system.out.println("p="+p);
        // Error p is a private member of X. Not accessible outside X.
        system.out.println("q="+q);    // Protected is accessible
        system.out.println("r="+r);    // public is accessible
    }
}
//Save this as Y.java in Mypackage1 directory
```



Access protection: An example

```
// Defining package MyPackage1
package MyPackage1
public class X {
    int n = 1;
    private int p = 2;
    protected int q = 3;
    public int r = 4;
    // A constructor of the class protection
}
//Save this as X.java in
```

```
//Save this as A.java in Mypackage1 directory
class A {                // class with default protection
    A() {                // default constructor with default access
        X x = new X() // create an object of class X
        System.out.println("Same package constructor ....");
        System.out.println("n from A"+x.n);
        // Default variable is accessible in the same package
        System.out.println("p from A"+x.p); // Error
        System.out.println("q from A"+x.q); // Error protection
        System.out.println("r from A"+x.r); // OK: public
    }
}
```



Access protection: An example

```
// Defining package MyPackage1
package MyPackage1
public class X {
    int n = 1;
    private int p = 2;
    protected int q = 3;
    public int r = 4;
    // A constructor of the class protection
}
//Save this as X.java in Mypackage1 directory
```

```
//Save this as 2.java in Mypackage2 directory
package MyPackage1;
class Z extends MyPackage1.X {
    Z() {
        System.out.println("I am constructor from class Z:");
        System.out.println("n from Z"+n); // Error:
        // Default is not accessible outside its package.
        System.out.println("p from Z"+p); // Error : private of X
        System.out.println("q from Z"+q);
        // Protected member is accessible by inheritance
        System.out.println("r from Z"+r);
        // public is accessible
    }
}
```



Access protection: An example

```
//Save this as B.java in Mypackage2 directory
class B {                                // class with default protection
    B() {                                // default constructor with default access
        MyPackagel.X x = new MyPackagel.X()
        // create an object of class X
        System.out.println("I am constructor from class B of MyPackage2");
        System.out.println("n from B of myPackage2"+x.n);
        // default variable but is not accessible in this package
        System.out.println("p from B of myPackage2"+x.p);
        // Error
        System.out.println("q from B of myPackage2"+x.q);
        // Error protection
        System.out.println("r from A of myPackage2"+x.r);
    }
}
```



Access protection: An example

```
//This is the demo of MyPackage1
package MyPackage1;
public class Demo1{
    public static void main(String args[])
    {
        X x1 = new X();
        Y y1 = new Y();
        A a1 = new A();
    }
}
```

```
//This is the demo of MyPackage2
package MyPackage2;
public class Demo2{
    public static void main(String args[])
    {
        Z z2 = new Z();
        B a2 = new B();
    }
}
```